FM

FLOWMATRIX.DEV

Premium Workflow

A practical, step-by-step playbook

Webhook Event Routing

Route incoming webhooks reliably: verify signatures, deduplicate events and forward them to the right services.

ENGINEERING ADVANCED ABOUT 2 HOURS SETUP

OVERVIEW

Webhook Event Routing

Route incoming webhooks reliably: verify signatures, deduplicate events and forward them to the right services.

CATEGORY	DIFFICULTY	DURATION
Engineering	**Advanced**	**About 2 hours setup**

Use case

Platforms integrating with multiple third-party services

Tools used in this workflow

Cloudflare Workers API · Supabase · n8n · Resend Email API

STEP BY STEP

How to build it

1 Receive webhook payloads

Empfange Webhook-Payloads von verschiedenen Quellen über HTTP POST-Anfragen und stelle sicher, dass die Daten korrekt und vollständig sind.

WAS DU TUST

1. Erstelle einen neuen Cloudflare Worker unter 'Workers' > 'Create a Worker'.
2. Definiere einen HTTP POST-Endpunkt im Worker-Skript, um die Payloads zu empfangen.
3. Logge die empfangenen Daten zur Überprüfung der Vollständigkeit.
4. Speichere die empfangenen Payloads in einer Supabase-Datenbanktabelle.

BEFEHLE & CODE

Cloudflare Worker erstellen

```
addEventListener('fetch', event =>
{ event.respondWith(handleRequest(event.request)) })
```

KONFIGURATION

SUPABASE_URL
<Deine Supabase URL> – Die URL deiner Supabase-Instanz.

SUPABASE_ANON_KEY
<Dein Supabase Anon Key> – Der anonyme Schlüssel für den Zugriff auf die Supabase-Datenbank.

CHECKLISTE

[ ] Cloudflare Worker ist aktiv und erreichbar.

[ ] Webhook-Payloads werden korrekt in der Supabase-Datenbank gespeichert.

ERGEBNIS

Webhook-Payloads werden erfolgreich empfangen und gespeichert.

2 Verify signatures

Überprüfe die Signaturen der empfangenen Webhook-Payloads, um die Authentizität der Daten sicherzustellen.

WAS DU TUST

1. Implementiere eine Funktion im Cloudflare Worker, die die Signatur der Payload überprüft.
2. Vergleiche die empfangene Signatur mit der erwarteten Signatur basierend auf einem geheimen Schlüssel.
3. Logge das Ergebnis der Signaturüberprüfung für jede Payload.
4. Leite nur Payloads mit gültiger Signatur zur weiteren Verarbeitung weiter.

BEFEHLE & CODE

Signaturüberprüfung

```
function verifySignature(payload, signature, secret) { /* Implementierung */ }
```

KONFIGURATION

SECRET_KEY
<Geheimer Schlüssel> – Der geheime Schlüssel zur Signaturüberprüfung.

CHECKLISTE

[ ] Signaturüberprüfung ist implementiert und getestet.

[ ] Nur Payloads mit gültiger Signatur werden weitergeleitet.

ERGEBNIS

Nur authentische Webhook-Payloads werden zur Verarbeitung zugelassen.

3

Deduplicate events

Stelle sicher, dass keine doppelten Ereignisse verarbeitet werden, indem du empfangene Webhooks auf Eindeutigkeit überprüfst.

WAS DU TUST

1. Füge jedem empfangenen Webhook ein eindeutiges Ereignis-ID-Feld hinzu.
2. Speichere die Ereignis-IDs in einer separaten Tabelle in Supabase.
3. Überprüfe bei jedem neuen Webhook, ob die Ereignis-ID bereits existiert.
4. Verarbeite nur Webhooks mit neuen, eindeutigen Ereignis-IDs.

CHECKLISTE

[ ] Ereignis-IDs werden korrekt generiert und gespeichert.

[ ] Doppelte Ereignisse werden erfolgreich erkannt und ignoriert.

ERGEBNIS

Nur einzigartige Webhook-Ereignisse werden verarbeitet.

4 Route to services

Leite die verifizierten und deduplizierten Payloads an die entsprechenden Backend-Services weiter.

WAS DU TUST

1. Erstelle Routing-Regeln basierend auf dem Inhalt der Payload.
2. Nutze n8n, um die Daten an verschiedene APIs oder interne Systeme zu senden.
3. Konfiguriere n8n-Workflows, um die Payloads korrekt zu verarbeiten.
4. Teste die Routing-Logik mit verschiedenen Payload-Beispielen.

KONFIGURATION

N8N_WEBHOOK_URL
<Dein n8n Webhook URL> – Die URL des n8n-Webhooks für die Weiterleitung.

CHECKLISTE

[ ] Routing-Regeln sind korrekt implementiert.
[ ] Payloads werden erfolgreich an die richtigen Services weitergeleitet.

ERGEBNIS

Payloads werden zuverlässig an die vorgesehenen Services weitergeleitet.

5 Alert on failures

Löse Alarme aus, um das Team im Falle von Fehlern während der Verarbeitung der Webhook-Payloads zu benachrichtigen.

WAS DU TUST

1. Implementiere Fehlerüberwachung im Cloudflare Worker.
2. Nutze die Resend Email API, um Benachrichtigungen bei Fehlern zu versenden.
3. Konfiguriere die E-Mail-Vorlage für Fehlerbenachrichtigungen.
4. Teste die Fehleralarme, indem du absichtlich Fehler in der Verarbeitung erzeugst.

KONFIGURATION

RESEND_API_KEY
<Dein Resend API Key> – API-Schlüssel für den Zugriff auf die Resend Email API.
ALERT_EMAIL
<Benachrichtigungs-E-Mail> – Die E-Mail-Adresse, an die Fehlerbenachrichtigungen gesendet werden.

CHECKLISTE

[ ] Fehlerüberwachung ist aktiv und funktioniert.
[ ] Fehlerbenachrichtigungen werden korrekt gesendet.

ERGEBNIS

Das Team wird bei Fehlern in der Webhook-Verarbeitung benachrichtigt.